

Codalyra-AI

Development Roadmap & Future Plan

Multi-Agent PR Review Pipeline

7 Phases | 25+ Features | Full-Stack Improvements

Author: Hamza Haji (HMZ-H)

github.com/HMZ-H/codalyra-ai

Current State

Codalyra-AI currently has a working multi-agent code review pipeline with 5 AI agents (logic, security, performance, quality, synthesis), static validators, a baseline comparison system, Celery-based parallel execution, JWT authentication, and a React frontend with pipeline visualization and score gauges.

The following roadmap outlines the features and improvements planned across 7 phases, ordered by priority and impact.

Phase 1: GitHub Integration

Priority: HIGH - Core Feature

- **GitHub OAuth Login**

Replace email/password auth with GitHub OAuth. Users log in with their GitHub account, giving the app access to their repositories.

Files: `backend/app/core/oauth.py`, `backend/app/api/v1/auth.py`

- **Repository Connection**

Allow users to connect GitHub repositories to projects. Fetch repo metadata, branches, and recent PRs via the GitHub API.

Files: `backend/app/services/github_service.py`, `backend/app/workers/github_tasks.py`

- **PR Webhook Integration**

Set up GitHub webhooks to automatically trigger multi-agent reviews when a PR is opened or updated. The diff is fetched directly from the GitHub API instead of manual pasting.

Files: `backend/app/api/v1/webhooks.py`, `backend/app/services/webhook_service.py`

- **Post Review Comments to PR**

After the review completes, post the findings as inline comments on the GitHub PR. Each finding maps to a specific file and line in the diff.

Files: `backend/app/services/github_comment_service.py`

- **PR List & Status Dashboard**

Show connected PRs on the frontend with review status, score, and link to GitHub. Allow re-triggering reviews on updated PRs.

Files: `frontend/src/pages/PullRequests.jsx`

Phase 2: Real-Time & UX Improvements

Priority: HIGH - User Experience

- **WebSocket Real-Time Updates**

Replace the 3-second polling on ReviewDetail with WebSocket connections. Each agent sends live status updates as it starts, completes, or fails. Much smoother UX.

Files: `backend/app/api/v1/ws.py`, `frontend/src/hooks/useReviewSocket.js`

- **Agent Log Streaming**

Stream agent thinking/trajectory logs in real-time during review. Users can watch each agent's reasoning process live, similar to how ChatGPT shows thinking.

Files: `frontend/src/components/AgentLogStream.jsx`

- Diff Viewer with Inline Annotations

Replace the plain text diff display with a proper syntax-highlighted diff viewer. Findings are shown as inline annotations on the relevant lines.

Files: `frontend/src/components/DiffViewer.jsx`

- Dark/Light Theme Toggle

Add a theme toggle to the frontend. The current UI is dark-only.

Files: `frontend/src/context/ThemeContext.jsx`, `frontend/src/styles/themes.css`

Phase 3: Security & Auth Hardening

Priority: HIGH - Critical Fixes

- Ownership Checks on All Endpoints

Add authorization checks so users can only access their own projects, reviews, and runs. Currently any authenticated user can read any review.

Files: `backend/app/core/permissions.py`, `backend/app/api/v1/reviews.py`

- Diff Size Limit & Validation

Add a max character limit on `diff_content` (e.g., 50,000 chars). Large diffs should be rejected with a clear error, or chunked for processing.

Files: `backend/app/schemas/review.py`

- Rate Limiting

Add rate limiting on review creation to prevent abuse. Limit per user per hour.

Files: `backend/app/middleware/rate_limit.py`

- API Key Management

Allow users to bring their own Gemini API key stored encrypted in the database, instead of using a shared server key.

Files: `backend/app/api/v1/settings.py`, `backend/app/database/models/user_settings.py`

Phase 4: Analytics & Review History

Priority: MEDIUM - Product Value

- Review History Dashboard

Show all past reviews with filterable list: by project, date range, score range, status. Includes pagination and search.

Files: `frontend/src/pages/ReviewHistory.jsx`

- Quality Score Trends

Line chart showing quality scores over time per project. Helps teams track if code quality is improving or degrading.

Files: `frontend/src/components/ScoreTrendChart.jsx`

- Finding Category Analytics

Bar/pie charts showing most common finding categories across reviews. Helps identify recurring patterns (e.g., team keeps introducing SQL injection).

Files: `frontend/src/components/CategoryBreakdown.jsx`

- Agent Performance Metrics

Track each agent's precision over time. How often do its findings survive synthesis? Which agent contributes the most unique findings?

Files: `backend/app/services/analytics_service.py`

- Export Reports

Allow downloading review reports as PDF or Markdown. Useful for compliance, audits, or sharing with non-technical stakeholders.

Files: `backend/app/services/export_service.py`, `backend/app/api/v1/exports.py`

Phase 5: Multi-Language & Smarter Agents

Priority: MEDIUM - Core Capability

- JavaScript/TypeScript Static Validator

Add regex patterns for JS/TS: `eval()`, `innerHTML`, prototype pollution, `console.log` in production, unused imports, any type abuse.

Files: `backend/app/validators/javascript_validator.py`

- Go Static Validator

Add patterns for Go: unchecked errors, goroutine leaks, defer in loops, `sql.Open` without `Close`, hardcoded credentials.

Files: `backend/app/validators/go_validator.py`

- Language Auto-Detection

Detect the programming language from the diff file extensions and route to the appropriate static validator automatically.

Files: `backend/app/validators/detector.py`

- Configurable Agent Prompts

Allow users to customize agent prompts per project. A team might want stricter security rules or domain-specific quality checks.

Files: `backend/app/database/models/agent_config.py`

- Multi-Model Support

Support switching between LLM providers (Gemini, Claude, GPT-4) per agent. Allow benchmarking different models against each other.

Files: `backend/app/ai/providers/`

Phase 6: Testing & DevOps

Priority: MEDIUM - Engineering Quality

- Unit Tests for AI Layer

Mock LLM responses and test the full pipeline: prompt construction, JSON parsing, finding deduplication, scoring, comparison logic.

Files: `backend/app/tests/test_ai/`

- Integration Tests for Review Pipeline

End-to-end tests that create a review, run mocked agents, trigger synthesis, and verify the final report structure.

Files: `backend/app/tests/test_reviews/`

- Frontend Tests

Component tests for `ReviewDetail`, `ReviewSubmit`, `Dashboard` using React Testing Library.

Files: `frontend/src/__tests__/`

- Full Docker Compose

Single docker compose up runs everything: API, Celery workers, frontend, PostgreSQL, Redis. Add health checks and proper networking.

Files: `docker-compose.yml`, `Dockerfile`, `frontend/Dockerfile`

- CI/CD Pipeline

GitHub Actions workflow: lint (ruff), test (pytest), build frontend, and optionally deploy to a staging environment.

Files: `.github/workflows/ci.yml`

Phase 7: Advanced Features

Priority: LOW - Future Vision

- Auto-Fix Suggestions

Beyond just identifying issues, generate actual code fix suggestions that users can apply with one click. Use the LLM to produce corrected code snippets.

Files: `backend/app/ai/fixer.py`

- Team & Organization Support

Multi-user teams with shared projects, role-based access (admin, reviewer, viewer), and team-wide analytics.

Files: `backend/app/database/models/team.py`, `backend/app/api/v1/teams.py`

- Slack/Discord Notifications

Send review results to a Slack or Discord channel when a review completes. Include score, critical findings count, and a link to the full report.

Files: `backend/app/services/notification_service.py`

- Custom Rule Engine

Let teams define custom regex or AST-based rules that run alongside the built-in validators. Project-specific patterns like banned functions or required imports.

Files: `backend/app/validators/custom_rules.py`

- Learning from Feedback

Track which findings users dismiss vs accept. Use this data to fine-tune agent prompts over time, reducing false positives per project.

Files: `backend/app/services/feedback_service.py`

Summary

Phase	Focus Area	Tasks	Priority
Phase 1	GitHub Integration	5 tasks	HIGH
Phase 2	Real-Time & UX	4 tasks	HIGH
Phase 3	Security & Auth	4 tasks	HIGH
Phase 4	Analytics & History	5 tasks	MEDIUM
Phase 5	Multi-Language & Smarter Agents	5 tasks	MEDIUM
Phase 6	Testing & DevOps	5 tasks	MEDIUM
Phase 7	Advanced Features	5 tasks	LOW

Total: 33 tasks across 7 phases

Recommended approach: Complete Phases 1-3 first (GitHub integration, real-time updates, and security fixes). These transform Codalyra-AI from a demo into a usable product. Phases 4-5 add product depth. Phase 6 ensures engineering quality. Phase 7 is the long-term vision.